

目录

一、Layouts（布局）	4
1. Vertical Layout（垂直布局）	4
2. Horizontal Layout（水平布局）	4
3. Grid Layout（网格布局）	4
4. Form Layout（表单布局）	5
二、Spacers（间隔）	5
1. Horizontal Spacer（水平间隔）	5
2. Vertical Spacer（垂直间隔）	5
三、Buttons（按钮）	5
1. Push Button（普通按钮）	6
2. Tool Button（工具按钮）	6
3. Radio Button（单选按钮）	6
4. Check Box（复选框）	6
5. Command Link Button（命令链接按钮）	7
6. Dialog Button Box（对话框按钮盒）	7
四、Item Views (Model-Based)（基于模型的项视图）	8
1. List View（列表视图）	8
2. Tree View（树视图）	8
3. Table View（表格视图）	8
4. Column View（列视图）	9
5. Undo View（撤销视图）	9
五、Item Widgets (Item-Based)（基于项的控件）	9
1. List Widget（列表控件）	9
2. Tree Widget（树控件）	9
3. Table Widget（表格控件）	10
六、Containers（容器）	11
1. Group Box（分组框）	11
2. Scroll Area（滚动区域）	11
3. Tool Box（工具箱）	11

4. Tab Widget（标签页控件）	12
5. Stacked Widget（堆叠控件）	12
6. Frame（框架）	12
7. Widget（基础控件）	12
8. MDI Area（多文档界面区域）	13
9. Dock Widget（停靠控件）	13
10. QAxWidget（ActiveX 控件）	13
七、Input Widgets（输入控件）	14
1. Combo Box（组合框）	14
2. Font Combo Box（字体组合框）	14
3. Line Edit（行编辑）	14
4. Text Edit（文本编辑）	15
5. Plain Text Edit（纯文本编辑）	15
6. Spin Box（整数微调框）	15
7. Double Spin Box（浮点数微调框）	15
8. Time Edit（时间编辑）	16
9. Date Edit（日期编辑）	16
10. Date/Time Edit（日期时间编辑）	16
11. Dial（刻度盘）	16
12. Horizontal Scroll Bar（水平滚动条）	17
13. Vertical Scroll Bar（垂直滚动条）	17
14. Horizontal Slider（水平滑块）	17
15. Vertical Slider（垂直滑块）	17
16. Key Sequence Edit（快捷键编辑）	18
八、Display Widgets（显示控件）	19
1. Label（标签）	19
2. Text Browser（文本浏览器）	19
3. Graphics View（图形视图）	19
4. Calendar Widget（日历控件）	20
5. LCD Number（LCD 数字显示）	20

6. Progress Bar（进度条）	20
7. Horizontal Line（水平线）	20
8. Vertical Line（垂直线）	21
9. OpenGL Widget（OpenGL 控件）	21
10. QQuickWidget（QML 嵌入控件）	21

一、Layouts（布局）

布局管理器是 Qt 界面开发的核心组件，用于自动管理控件的位置和大小，确保界面在不同分辨率和窗口尺寸下都能保持良好的显示效果。布局管理器接管了控件的几何管理，使开发者无需手动计算每个控件的坐标。

1. Vertical Layout（垂直布局）

QVBoxLayout

功能说明：垂直布局将控件按从上到下的顺序垂直排列。它是 QVBoxLayout 的子类，专门处理垂直方向的控件排列，自动分配每个控件的高度。

典型应用场景：适用于构建纵向列表、菜单、设置面板、表单输入区域等需要垂直排列控件的场景；常用于按钮组、输入框序列等需要按列展示的界面。

基本使用要点：核心方法：addWidget() 添加控件，addLayout() 嵌套布局，addStretch() 添加伸缩空间；可通过 setSpacing() 设置控件间距，setContentsMargins() 设置边距；使用 setStretch() 设置拉伸因子控制控件高度比例。

2. Horizontal Layout（水平布局）

QHBoxLayout

功能说明：水平布局将控件按从左到右的顺序水平排列。与垂直布局对称，专门处理水平方向的控件排列，自动分配每个控件的宽度。

典型应用场景：适用于工具栏、导航栏、水平按钮组、搜索框组合、状态栏等需要水平排列控件的场景；常用于单行控件的布局管理。

基本使用要点：API 与垂直布局完全一致，仅排列方向不同；重点关注 stretch 因子的使用来控制宽度分配；可与垂直布局嵌套实现复杂的网格状布局。

3. Grid Layout（网格布局）

QGridLayout

功能说明：网格布局将可用空间划分为行和列，形成二维网格，控件可以放置在指定的单元格中，支持跨越多行或多列。

典型应用场景：适用于表单、数据输入界面、计算器键盘、控制面板等需要规整网格排列的复杂布局；是最灵活的布局管理器。

基本使用要点：使用 addWidget(widget, row, col, rowspan, colspan) 指定位置和跨度；通过 setRowStretch() 和 setColumnStretch() 设置行列的拉伸比例；setSpacing() 设置单元格间距，支持单元格合并。

4. Form Layout（表单布局）

QFormLayout

功能说明：表单布局专门用于创建标签-输入框对的两列布局，左列显示标签，右列显示输入控件，自动对齐和管理表单字段。

典型应用场景：适用于用户注册/登录表单、设置对话框、属性编辑面板、数据录入界面等标准表单场景；是构建输入表单的最佳选择。

基本使用要点：使用 `addRow(label, field)` 添加一行表单项；支持 `setLabelAlignment()` 设置标签对齐方式；可通过 `setRowWrapPolicy()` 控制长字段的换行行为；自动处理不同平台的表单布局规范。

二、Spacers（间隔）

间隔控件是布局管理器的辅助组件，用于在布局中创建空白区域，调整控件之间的间距，控制控件的对齐方式，实现响应式的界面布局。

1. Horizontal Spacer（水平间隔）

Horizontal Spacer (QSpacerItem)

功能说明：在水平方向上创建可伸缩的空白区域，本质是 `QSpacerItem` 的水平配置版本，水平方向具有 `Expanding` 大小策略，垂直方向为 `Minimum`。

典型应用场景：用于在水平布局中分隔控件组、实现左右对齐（如工具栏左对齐按钮右对齐）、吸收多余水平空间、防止控件被拉伸变形。

基本使用要点：大小策略为水平 `Expanding`，垂直 `Minimum`；通过 `changeSize()` 可自定义尺寸策略；在 Qt Designer 中直接拖放使用，代码中使用 `addSpacerItem()` 添加；多个 `Spacer` 按比例分配空间。

2. Vertical Spacer（垂直间隔）

Vertical Spacer (QSpacerItem)

功能说明：在垂直方向上创建可伸缩的空白区域，垂直方向具有 `Expanding` 大小策略，水平方向为 `Minimum`，像弹簧一样吸收垂直方向的多余空间。

典型应用场景：用于在垂直布局中上下分隔控件组、将控件推到窗口顶部或底部、在表单中创建分组间距、实现垂直方向的自适应布局。

基本使用要点：大小策略为垂直 `Expanding`，水平 `Minimum`；与 `addStretch()` 功能类似但更灵活；可设置固定高度或可伸缩；常用于对话框底部按钮与内容之间的间距。

三、Buttons（按钮）

按钮类控件是用户交互的核心组件，提供各种点击、选择、确认等操作方式，继承自 `QAbstractButton` 抽象基类。

1. Push Button（普通按钮）

QPushButton

功能说明：最常用的命令按钮，点击后触发一个动作。支持显示文本和图标，可设置为默认按钮或自动重复按钮，是所有按钮中功能最全面的。

典型应用场景：适用于确认、取消、保存、打开、关闭等所有需要触发即时操作的场景；是对话框和主窗口中最常见的交互控件。

基本使用要点：核心信号：clicked() 点击信号；setText() 设置文本，setIcon() 设置图标；setDefault() 设置为对话框默认按钮；setAutoRepeat() 启用自动重复；setCheckable() 设置为可切换状态。

2. Tool Button（工具按钮）

QToolButton

功能说明：专门用于工具栏的按钮，通常显示图标而非文本，支持弹出菜单、自动升起外观、多种工具按钮样式，比普通按钮更紧凑。

典型应用场景：适用于工具栏、工具面板、快捷操作栏；常用于需要图标为主、文本为辅的操作按钮；支持菜单弹出功能。

基本使用要点：setToolButtonStyle() 设置图标文本显示模式；setPopupMode() 设置菜单弹出方式；setAutoRaise() 启用自动升起效果；常与 QToolBar 配合使用；支持箭头指示有子菜单。

3. Radio Button（单选按钮）

QRadioButton

功能说明：单选按钮用于在一组互斥选项中选择唯一一项。同一按钮组中只能有一个按钮被选中，选中后自动取消同组其他按钮的选中状态。

典型应用场景：适用于性别选择、模式切换、选项互斥的设置、单选问卷等多选一场景；必须成组使用才有意义。

基本使用要点：需放入 QButtonGroup 管理互斥性；isChecked() 检查是否选中；toggled(bool) 状态变化信号；setAutoExclusive() 控制排他性；文本中&符号可设置快捷键。

4. Check Box（复选框）

QCheckBox

功能说明：复选框用于开启/关闭某个选项，支持独立选择或多选。与单选按钮不同，复选框之间互不影响，可以同时选中多个。

典型应用场景：适用于功能开关、权限设置、多选项选择、同意条款、特性启用等 yes/no 或多选场景。

基本使用要点：支持三态模式（选中/未选中/半选中）；`stateChanged(int)` 状态变化信号；`setTristate()` 启用三态；`checkState()` 获取三态值；`isChecked()` 获取二态选中状态。

5. Command Link Button（命令链接按钮）

`QCommandLinkButton`

功能说明：Vista 风格的命令链接按钮，外观类似平面按钮但带有箭头图标，支持主标题和描述文本两行显示，用于向导式交互。

典型应用场景：适用于安装向导、配置向导、对话框中的选项选择、下一步操作导航；替代单选按钮提供更直观的选项选择。

基本使用要点：继承自 `QPushButton`，`setDescription()` 设置描述文本；设计用于互斥选项选择，类似单选按钮的作用；默认带有向右箭头图标；`flat` 外观更现代。

6. Dialog Button Box（对话框按钮盒）

`QDialogButtonBox`

功能说明：对话框按钮盒是标准按钮的容器，自动按照平台规范排列 OK、Cancel、Yes、No 等标准按钮，确保按钮顺序符合操作系统习惯。

典型应用场景：适用于所有对话框的底部按钮区、消息框、属性对话框、确认对话框；确保跨平台一致性的最佳实践。

基本使用要点：`addButton()` 添加标准按钮或自定义按钮；`standardButtons()` 设置标准按钮组合；`accepted()` 和 `rejected()` 信号；自动处理按钮布局和顺序；支持 `Orientation` 设置水平/垂直排列。

四、Item Views (Model-Based)（基于模型的项视图）

基于模型的项视图采用 MVC（模型-视图-控制器）架构，将数据存储与数据显示分离。视图负责显示，模型负责数据管理，代理负责编辑。这种架构支持大数据量高效显示，适合复杂数据展示。

1. List View（列表视图）

QListView

功能说明：以列表形式显示模型中的数据项，支持列表模式和图标模式，可自定义项的显示方式和选择模式，支持大数据量高效渲染。

典型应用场景：适用于文件列表、选项列表、图片缩略图列表、数据项选择等一维数据展示；大数据量列表的首选控件。

基本使用要点：setModel() 设置数据模型；setViewMode() 设置 ListMode 或 IconMode；setMovement() 控制项是否可拖动；支持自定义委托绘制项；仅渲染可见项，性能优异。

2. Tree View（树视图）

QTreeView

功能说明：以树形层次结构显示数据，支持父子节点的展开和折叠，展示数据的层级关系，是展示结构化数据的标准控件。

典型应用场景：适用于文件系统目录、组织结构、分类菜单、XML/JSON 结构展示、注册表编辑器等具有层级关系的数据。

基本使用要点：setModel() 设置标准项目模型或自定义模型；setHeaderHidden() 隐藏表头；expand()/collapse() 控制节点展开；支持多列树视图；延迟加载提升性能。

3. Table View（表格视图）

QTableView

功能说明：以二维表格形式显示数据，类似电子表格，支持行和列的操作，是展示结构化数据最强大的控件，支持排序、筛选、选择。

典型应用场景：适用于数据库表格、数据报表、电子表格、属性表格、多维数据展示等需要行列结构的场景；企业级数据展示首选。

基本使用要点：setModel() 设置表格模型；horizontalHeader()/verticalHeader() 获取表头；setSortingEnabled() 启用排序；setSelectionMode() 设置选择模式；支持单元格合并和自定义委托。

4. Column View（列视图）

QColumnView

功能说明：以多列级联的方式显示树形数据，每一列显示一个层级的内容，选择父项后右侧显示子项，类似 Mac OS X 的 Finder 视图。

典型应用场景：适用于文件系统浏览、分类导航、层级数据探索、多步骤选择等需要逐级深入查看层级数据的场景。

基本使用要点：setModel() 设置模型；createColumn() 创建新列；previewWidget 设置预览区域；适合层次较深但每层数据量不大的结构；提供直观的层级导航体验。

5. Undo View（撤销视图）

QUndoView

功能说明：显示 QUndoStack 中的命令历史，以列表形式展示所有可撤销/重做的操作，用户可点击跳转到任意历史状态。

典型应用场景：适用于图像编辑、文档编辑、CAD 软件等需要撤销/重做功能的应用；作为操作历史面板使用。

基本使用要点：setStack() 关联撤销栈；显示的命令来自 QUndoCommand 的 text()；点击列表项可跳转到对应历史状态；cleanChanged() 信号标识状态是否已保存；集成到 QDockWidget 中作为侧边面板。

五、Item Widgets (Item-Based)（基于项的控件）

基于项的控件是视图控件的便捷版本，内置了数据存储，无需单独创建模型，使用更简单。它们封装了模型/视图架构，适合简单数据场景，但大数据量性能不如纯视图。

1. List Widget（列表控件）

QListWidget

功能说明：便捷的列表控件，内置数据模型，直接操作 QListWidgetItem 项，无需理解模型/视图架构即可使用，API 简单直观。

典型应用场景：适用于简单列表、选择框、任务列表、小数据量选项展示等不需要复杂数据处理的场景；快速开发的首选。

基本使用要点：addItem()/addItems() 添加项；setCurrentRow() 设置当前选中；itemClicked() 信号；支持拖放排序；item() 获取指定项；简单场景下比 QListView 更易用。

2. Tree Widget（树控件）

QTreeWidget

功能说明：便捷的树形控件，内置树形数据模型，直接操作 QTreeWidgetItem 节点，支持多列表头，使用简单无需自定义模型。

典型应用场景：适用于简单目录树、配置树、分类结构、属性树等层级不深、数据量不大的树形数据展示。

基本使用要点：addTopLevelItem() 添加顶层节点；QTreeWidgetItem 添加子节点；setColumnCount() 设置列数；setHeaderLabels() 设置表头；currentItemChanged() 信号；适合快速开发。

3. Table Widget（表格控件）

QTableWidget

功能说明：便捷的表格控件，内置表格数据模型，直接操作单元格，支持行列管理，无需创建自定义模型，开发效率高。

典型应用场景：适用于简单数据表格、属性编辑、参数配置、小数据量报表等不需要复杂数据处理的表格场景。

基本使用要点：setRowCount()/setColumnCount() 设置行列数；setItem() 设置单元格内容；item() 获取单元格；cellClicked() 信号；支持单元格合并；简单表格开发首选。

六、Containers（容器）

容器控件用于包含和管理其他控件，提供分组、滚动、分页、停靠等功能，是构建复杂界面的基础框架组件。

1. Group Box（分组框）

QGroupBox

功能说明：带标题的分组框，用于视觉上和逻辑上分组相关控件，提供边框和标题，可设置为可勾选以启用/禁用整组控件。

典型应用场景：适用于单选按钮组、复选框组、相关设置项分组、表单字段分组等需要将相关控件组织在一起的场景。

基本使用要点：setTitle() 设置标题；setCheckable() 设置为可勾选；toggled(bool) 信号控制子控件启用状态；setFlat() 设置扁平样式；是最常用的分组容器。

2. Scroll Area（滚动区域）

QScrollArea

功能说明：提供滚动视口，当子控件内容超出显示区域时自动显示滚动条，支持水平和垂直滚动，确保大内容也能完整显示。

典型应用场景：适用于大图片显示、长表单、设置页面、帮助文档、内容可能超出窗口大小的任何场景。

基本使用要点：setWidget() 设置要滚动的子控件；setWidgetResizable() 自动调整子控件大小；ensureVisible() 滚动到指定位置；horizontalScrollBar()/verticalScrollBar() 获取滚动条；支持自定义滚动条策略。

3. Tool Box（工具箱）

QToolBox

功能说明：列状抽屉式容器，每个页面可展开/折叠，类似 Photoshop 的工具箱，一次只展开一个页面，节省垂直空间。

典型应用场景：适用于工具面板、分类工具栏、属性分组面板、功能模块分组等需要分类展示且空间有限的场景。

基本使用要点：addItem() 添加抽屉页；setCurrentIndex() 切换当前页；currentChanged() 信号；itemText()/setItemText() 设置页标题；支持页图标；经典的分类收纳 UI。

4. Tab Widget（标签页控件）

QTabWidget

功能说明：标签页容器，通过顶部或侧边的标签切换不同页面，是最常用的多页面容器，每个标签对应一个独立的功能页面。

典型应用场景：适用于设置对话框、属性面板、多文档界面、功能模块切换等需要在同一窗口切换多种内容的场景。

基本使用要点：addTab() 添加标签页；setTabPosition() 设置标签位置；currentChanged() 信号；setTabsClosable() 支持关闭标签；setMovable() 支持标签拖动排序；功能最完善的多页控件。

5. Stacked Widget（堆叠控件）

QStackedWidget

功能说明：堆叠式容器，多个子控件堆叠在一起，同一时间只显示一个，没有自带的切换控件，需要外部控制切换。

典型应用场景：适用于向导界面、自定义多页界面、根据条件显示不同内容、状态切换界面等需要程序控制页面切换的场景。

基本使用要点：addWidget() 添加页面；setCurrentIndex()/setCurrentWidget() 切换页面；currentChanged() 信号；没有默认切换 UI，完全由程序控制；是实现向导的基础控件。

6. Frame（框架）

QFrame

功能说明：基础框架控件，提供边框和形状，可作为其他控件的容器或分隔符，支持多种边框样式和阴影效果。

典型应用场景：适用于控件分组的视觉边框、水平线/垂直线分隔、面板容器、状态指示器、自定义绘制基础等。

基本使用要点：setFrameShape() 设置框架形状；setFrameShadow() 设置阴影效果；setLineWidth() 设置线宽；是 QLabel 和 QAbstractScrollArea 的基类；可作为简单容器使用。

7. Widget（基础控件）

QWidget

功能说明：所有用户界面对象的基类，是空白的通用容器，可作为自定义控件的基类或其他控件的容器，是 Qt 界面的基础构建块。

典型应用场景：所有控件的基类；自定义控件的父类；子控件的容器；顶层窗口；绘制区域；是 Qt 中最基础最重要的类。

基本使用要点：setLayout() 设置布局；show()/hide() 显示隐藏；setWindowTitle() 设置窗口标题；setParent() 设置父控件；paintEvent() 自定义绘制；所有控件都继承此类。

8. MDI Area（多文档界面区域）

QMdiArea

功能说明：多文档界面容器，管理多个子窗口，支持子窗口的级联、平铺、最大化、最小化等操作，实现传统 MDI 应用。

典型应用场景：适用于文字处理、图像编辑、IDE 等需要同时打开多个文档窗口的应用；传统桌面软件的经典界面。

基本使用要点：addSubWindow() 添加子窗口；cascadeSubWindows()/tileSubWindows() 排列窗口；activeSubWindow() 获取活动窗口；subWindowActivated() 信号；支持 TabbedView 模式。

9. Dock Widget（停靠控件）

QDockWidget

功能说明：可停靠的面板窗口，可停靠在主窗口四周或浮动为独立窗口，支持拖动、停靠、浮动、关闭，构建灵活的界面布局。

典型应用场景：适用于工具栏、属性面板、项目浏览器、输出控制台、工具选项板等需要灵活位置的辅助面板；IDE 类应用必备。

基本使用要点：setAllowedAreas() 设置允许停靠的区域；setFeatures() 设置停靠特性；dockLocationChanged() 信号；toggleViewAction() 获取切换显示的动作；与 QMainWindow 配合使用。

10. QAxWidget（ActiveX 控件）

QAxWidget

功能说明：ActiveX 控件容器，用于在 Qt 应用中嵌入和操作 Windows 的 ActiveX 控件和 COM 对象，实现与 Windows 组件的互操作。

典型应用场景：适用于嵌入 Office 控件、浏览器控件、媒体播放器、第三方 ActiveX 组件等 Windows 平台特有的功能集成。

基本使用要点：setControl() 加载 ActiveX 控件；dynamicCall() 调用 COM 方法；querySubObject() 获取子对象；仅 Windows 平台可用；需要 axcontainer 模块；注意安全和兼容性问题。

七、Input Widgets（输入控件）

输入控件用于接收用户的各种输入，包括文本、数字、日期、时间、范围值等，是用户数据输入的主要交互组件。

1. Combo Box（组合框）

QComboBox

功能说明：下拉选择框，结合了单行编辑框和下拉列表，用户可从预定义选项中选择或输入自定义内容，节省界面空间。

典型应用场景：适用于选项较多的下拉选择、历史记录选择、可编辑的选择输入、分类选择等需要从列表选择的场景。

基本使用要点：addItem()/addItems() 添加选项；currentText()/currentIndex() 获取当前选择；currentTextChanged() 信号；setEditable() 设置可编辑；setInsertPolicy() 控制插入策略。

2. Font Combo Box（字体组合框）

QFontComboBox

功能说明：专门的字体选择组合框，自动填充系统所有字体名称，并可预览字体效果，是字体选择的专用控件。

典型应用场景：适用于文字处理软件、富文本编辑器、设置对话框中的字体选择场景；比普通组合框更专业。

基本使用要点：currentFont() 获取当前字体；currentFontChanged() 信号；setFontFilters() 过滤字体类型；setWritingSystem() 设置书写系统；自动加载系统字体无需手动添加。

3. Line Edit（行编辑）

QLineEdit

功能说明：单行文本输入框，支持密码模式、输入验证、占位符文本、输入掩码、自动补全等功能，是最常用的输入控件。

典型应用场景：适用于用户名、密码、搜索框、文件名、短文本等单行输入场景；几乎每个表单都会用到。

基本使用要点：setText()/text() 设置获取文本；setEchoMode() 设置密码模式；setPlaceholderText() 设置占位符；setValidator() 设置验证器；textChanged()/editingFinished() 信号。

4. Text Edit（文本编辑）

QTextEdit

功能说明：富文本编辑器，支持 HTML 格式、图片、表格、多种字体颜色样式，是 Qt 中功能最强大的文本编辑控件。

典型应用场景：适用于富文本编辑、邮件编辑、帮助文档显示、HTML 内容编辑、带格式的文字处理等场景。

基本使用要点：setHtml()/setPlainText() 设置内容；toHtml()/toPlainText() 获取内容；textChanged() 信号；setReadOnly() 设置只读；支持撤销/重做、复制粘贴；可显示大文档。

5. Plain Text Edit（纯文本编辑）

QPlainTextEdit

功能说明：纯文本编辑器，专为大文档和快速输入优化，不支持富文本格式，但处理大文本性能远优于 QTextEdit，支持语法高亮。

典型应用场景：适用于代码编辑器、日志查看、大文本文件查看、纯文本笔记等不需要富文本的场景；代码编辑器的基础。

基本使用要点：setPlainText()/toPlainText() 设置获取文本；setLineWrapMode() 设置换行模式；textChanged() 信号；支持块高亮；可自定义语法高亮；高性能处理大文档。

6. Spin Box（整数微调框）

QSpinBox

功能说明：整数输入微调框，通过上下按钮或键盘输入整数值，支持范围限制、步长设置、前缀后缀显示，确保输入合法整数。

典型应用场景：适用于数量设置、参数调整、页码输入、年龄、数量等整数数值输入场景。

基本使用要点：setValue()/value() 设置获取值；setRange() 设置范围；setSingleStep() 设置步长；setPrefix()/setSuffix() 设置前后缀；valueChanged() 信号。

7. Double Spin Box（浮点数微调框）

QDoubleSpinBox

功能说明：浮点数输入微调框，支持小数输入，可设置小数位数，其他功能与整数微调框相同，确保合法浮点数输入。

典型应用场景：适用于价格、比例、精度参数、测量值等需要小数精度的数值输入场景。

基本使用要点：setDecimals() 设置小数位数；其他 API 与 QSpinBox 相同；支持科学计数法显示；可设置特殊值文本；高精度数值输入的标准控件。

8. Time Edit（时间编辑）

QTimeEdit

功能说明：时间输入编辑框，专门用于输入和编辑时间（时:分:秒），支持时间范围限制和格式设置，确保合法时间输入。

典型应用场景：适用于日程安排、闹钟设置、倒计时、时间参数配置等需要输入时间的场景。

基本使用要点：setTime()/time() 设置获取时间；setTimeRange() 设置范围；setDisplayFormat() 设置显示格式；timeChanged() 信号；支持 12/24 小时制。

9. Date Edit（日期编辑）

QDateEdit

功能说明：日期输入编辑框，专门用于输入和编辑日期（年-月-日），支持弹出日历选择，确保合法日期输入。

典型应用场景：适用于生日选择、日期筛选、日程安排、有效期设置等需要输入日期的场景。

基本使用要点：setDate()/date() 设置获取日期；setDateRange() 设置范围；setCalendarPopup() 启用日历弹出；dateChanged() 信号；支持多种日期格式。

10. Date/Time Edit（日期时间编辑）

QDateTimeEdit

功能说明：日期时间综合编辑框，同时编辑日期和时间，支持日历弹出，是 QDateEdit 和 QTimeEdit 的组合增强版。

典型应用场景：适用于时间戳、事件时间、记录创建时间、精确时间点等需要同时指定日期和时间的场景。

基本使用要点：setDateTime()/dateTime() 设置获取日期时间；其他 API 继承自 QDateEdit 和 QTimeEdit；支持日历弹出选择日期；完整的时间戳输入控件。

11. Dial（刻度盘）

QDial

功能说明：圆形刻度盘控件，类似旋钮或音量旋钮，通过旋转选择数值，提供直观的模拟调节体验。

典型应用场景：适用于音量调节、亮度调节、角度设置、速度控制等模拟量调节场景；提供更直观的交互体验。

基本使用要点：setValue()/value() 设置获取值；setRange() 设置范围；setNotchesVisible() 显示刻度；valueChanged() 信号；setWrapping() 设置是否循环；模拟旋钮的经典控件。

12. Horizontal Scroll Bar（水平滚动条）

QScrollBar

功能说明：水平滚动条控件，用于控制水平方向的滚动位置，也可作为数值范围输入控件，提供直观的比例调节。

典型应用场景：适用于内容滚动、数值范围调节、进度控制、缩放控制等需要水平方向滑动调节的场景。

基本使用要点：setValue()/value() 设置获取值；setRange() 设置范围；setPageStep() 设置页步长；valueChanged() 信号；sliderPressed()/sliderMoved()/sliderReleased() 信号。

13. Vertical Scroll Bar（垂直滚动条）

QScrollBar

功能说明：垂直滚动条控件，用于控制垂直方向的滚动位置，API 与水平滚动条完全相同，仅方向不同。

典型应用场景：适用于文档滚动、列表滚动、垂直数值调节、音量控制等需要垂直方向滑动调节的场景。

基本使用要点：API 与水平滚动条完全一致；orientation 属性区分方向；通常由滚动区域自动创建，也可单独作为调节控件使用；所有滚动视图的基础组件。

14. Horizontal Slider（水平滑块）

QSlider

功能说明：水平滑块控件，通过拖动滑块在轨道上移动来选择数值，比滚动条更简洁，专门用于数值输入，不用于滚动。

典型应用场景：适用于音量控制、亮度调节、透明度、缩放比例、进度设置等需要直观滑动调节的场景。

基本使用要点：setValue()/value() 设置获取值；setRange() 设置范围；setTickPosition() 设置刻度位置；valueChanged() 信号；setOrientation() 设置方向；最常用的数值调节控件。

15. Vertical Slider（垂直滑块）

QSlider

功能说明：垂直滑块控件，API 与水平滑块完全相同，仅方向为垂直，适合空间有限或需要垂直调节的场景。

典型应用场景：适用于混音器、均衡器、垂直空间有限的调节面板、音频处理软件等垂直排列的调节场景。

基本使用要点：API 与水平滑块完全一致；orientation 设置为 Vertical；常用于多轨道并排的音频调节；节省水平空间；视觉上更符合某些专业软件的习惯。

16. Key Sequence Edit（快捷键编辑）

`QKeySequenceEdit`

功能说明：快捷键序列编辑控件，专门用于捕获和编辑键盘快捷键，自动记录用户按下的组合键，防止无效快捷键。

典型应用场景：适用于软件快捷键设置、热键配置、自定义快捷键等需要用户设置键盘快捷键的场景。

基本使用要点：`setKeySequence()`/`keySequence()` 设置获取快捷键；`keySequenceChanged()` 信号；自动捕获按键组合；支持多键序列；防止冲突的专业控件。

八、Display Widgets（显示控件）

显示控件用于向用户展示信息，包括文本、图像、数据、状态等，通常不接收用户输入，是信息展示的核心组件。

1. Label（标签）

QLabel

功能说明：文本或图像显示控件，可显示纯文本、富文本、图片、动画，是最简单最常用的显示控件，也可作为其他控件的标签。

典型应用场景：适用于显示静态文本、图片、图标、动画、视频帧、字段标签等几乎所有信息展示场景；使用频率最高的控件之一。

基本使用要点：setText() 设置文本，支持 HTML 格式；setPixmap() 设置图片；setMovie() 播放 GIF 动画；setAlignment() 设置对齐；setScaledContents() 缩放图片；可作为伙伴控件设置快捷键。

2. Text Browser（文本浏览器）

QTextBrowser

功能说明：富文本浏览器，继承自 QTextEdit 但只读，支持超链接导航、历史记录、文档浏览，专门用于显示富文本文档。

典型应用场景：适用于帮助文档、README 显示、HTML 内容展示、日志查看、带链接的说明文档等只读富文本显示场景。

基本使用要点：setSource() 加载 HTML 文档；setHtml() 设置 HTML 内容；anchorClicked() 超链接点击信号；backward()/forward() 历史导航；openExternalLinks() 打开外部链接。

3. Graphics View（图形视图）

QGraphicsView

功能说明：2D 图形视图框架的视图组件，用于显示 QGraphicsScene 中的大量图形项，支持缩放、旋转、选择、交互。

典型应用场景：适用于 CAD 绘图、流程图、图表、游戏、图像编辑、2D 可视化等需要大量图形元素交互的场景；Qt 的 2D 绘图核心框架。

基本使用要点：setScene() 设置场景；setRenderHint() 设置渲染质量；scale()/rotate() 变换；fitInView() 自适应显示；mapToScene() 坐标转换；与 QGraphicsScene 和 QGraphicsItem 配合使用。

4. Calendar Widget（日历控件）

QCalendarWidget

功能说明：月历视图控件，显示一个月的日历，支持日期选择、导航、标记特殊日期，提供直观的日期选择界面。

典型应用场景：适用于日期选择、日程查看、日历应用、预约系统、日期筛选等需要可视化日历的场景。

基本使用要点：selectedDate() 获取选中日期；setSelectedDate() 设置日期；selectionChanged() 信号；setDateTextFormat() 标记特殊日期；setMinimumDate()/setMaximumDate() 设置范围。

5. LCD Number（LCD 数字显示）

QLCDNumber

功能说明：LCD 风格的数字显示控件，以七段数码管样式显示数字，支持整数和浮点数，提供复古的数字显示效果。

典型应用场景：适用于计时器、计数器、仪表显示、数值监控、仿真界面等需要突出显示数值的场景。

基本使用要点：display() 设置显示数值；setDigitCount() 设置位数；setMode() 设置进制（十进制/十六进制/八进制/二进制）；setSegmentStyle() 设置显示样式；overflow() 信号。

6. Progress Bar（进度条）

QProgressBar

功能说明：进度显示控件，水平或垂直显示操作进度，支持文本显示、不确定模式，直观展示任务完成情况。

典型应用场景：适用于文件复制、下载进度、加载进度、处理进度、安装进度等需要显示操作进度的场景。

基本使用要点：setValue()/value() 设置获取进度；setRange() 设置范围；setOrientation() 设置方向；setFormat() 设置显示文本；setBusyIndicator() 启用不确定模式。

7. Horizontal Line（水平线）

QFrame (HLine)

功能说明：水平分隔线，是 QFrame 的一种预设样式，用于在界面上创建水平分隔线，视觉上分隔不同区域。

典型应用场景：适用于分组控件之间的分隔、标题与内容的分隔、工具栏分隔、界面区域划分等视觉分隔场景。

基本使用要点：frameShape 设置为 HLine；setFrameShadow() 设置阴影效果；setLineWidth() 设置线宽；纯视觉装饰控件；改善界面布局的可读性。

8. Vertical Line（垂直线）

QFrame (VLine)

功能说明：垂直分隔线，是 QFrame 的一种预设样式，用于在界面上创建垂直分隔线，视觉上分隔不同区域。

典型应用场景：适用于并排控件之间的分隔、工具栏按钮组分隔、左右面板分隔、列之间的视觉分隔场景。

基本使用要点：frameShape 设置为 VLine；其他属性与水平线相同；纯视觉装饰；常用于水平布局中控件之间的视觉分隔；提升界面专业感。

9. OpenGL Widget（OpenGL 控件）

QOpenGLWidget

功能说明：OpenGL 渲染窗口，用于显示 3D 图形，集成 OpenGL 渲染到 Qt 界面，支持硬件加速的 3D 图形显示。

典型应用场景：适用于 3D 建模、科学可视化、游戏、CAD、虚拟现实、数据可视化等需要 3D 图形的场景。

基本使用要点：initializeGL()/resizeGL()/paintGL() 三个关键方法；makeCurrent()/doneCurrent() 管理上下文；支持 OpenGL ES；与 Qt 界面无缝集成；Qt5.4+ 推荐使用。

10. QQuickWidget（QML 嵌入控件）

QQuickWidget

功能说明：QML 界面嵌入控件，用于在 QWidget 应用中嵌入 Qt Quick QML 界面，实现 QWidget 与 QML 混合开发。

典型应用场景：适用于在传统 QWidget 应用中嵌入 QML 动画、QML 自定义控件、Qt Quick 界面，实现新旧技术混合开发。

基本使用要点：setSource() 加载 QML 文件；rootObject() 获取 QML 根对象；setResizeMode() 设置调整模式；需要 QtQuick 模块；实现 C++ 与 QML 交互的桥梁。